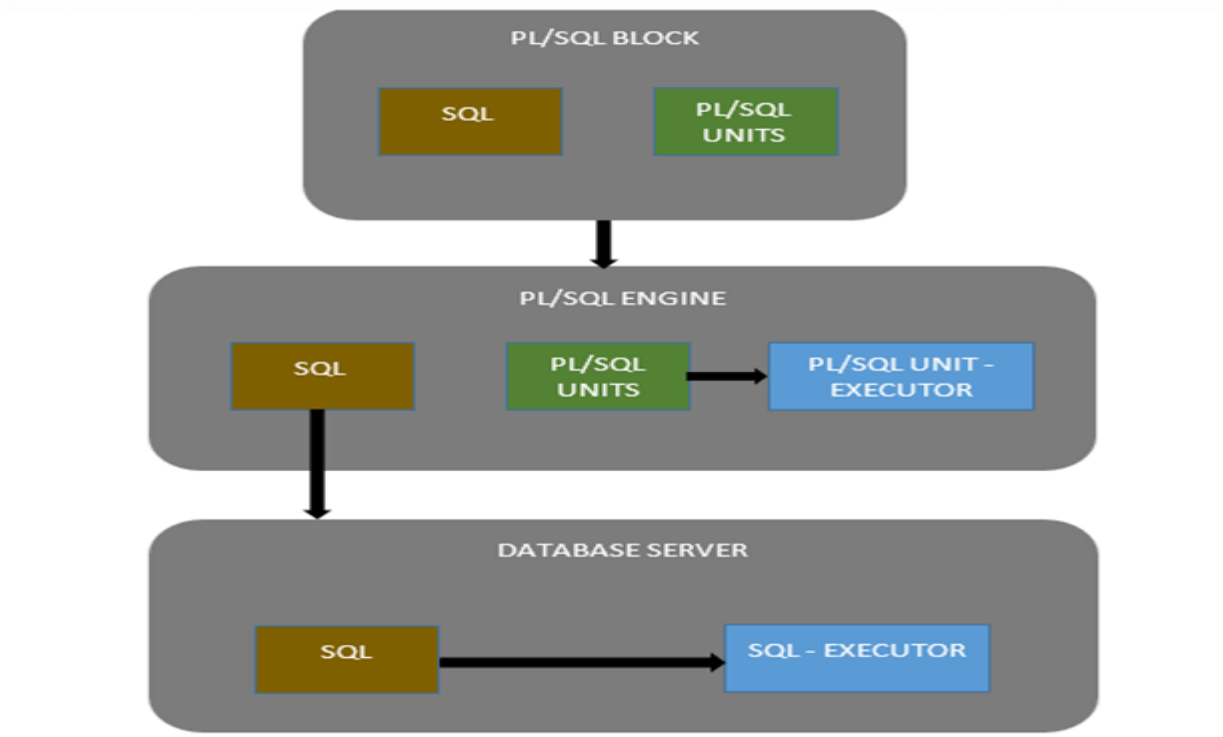
Architecture of PL/SQL

The Below PL/SQL Example is a pictorial representation of PL/SQL Architecture.



The PL/SQL architecture mainly consists of following three components:

1. PL/SQL Block
2. PL/SQL Engine
3. Database Server

PL/SQL block

- This is the component which has the actual PL/SQL code.
- This consists of different sections to divide the code logically (declarative section for declaring purpose, execution section for processing statements, exception handling section for handling errors)
- It also contains the SQL instruction that used to interact with the database server.
- All the PL/SQL units are treated as PL/SQL blocks, and this is the starting stage of the architecture which serves as the primary input.

Following are the different type of PL/SQL units.

- Anonymous Block
- Function
- Library
- Procedure

- Package Body
- Package Specification
- Trigger
- Type
- Type Body

PL/SQL Engine

- PL/SQL engine is the component where the actual processing of the codes takes place.
- PL/SQL engine separates PL/SQL units and SQL part in the input (as shown in the image below).
- The separated PL/SQL units will be handled by the PL/SQL engine itself.
- The SQL part will be sent to database server where the actual interaction with database takes place.
- It can be installed in both database server and in the application server.

Database Server

- This is the most important component of Pl/SQL unit which stores the data.
- The PL/SQL engine uses the SQL from PL/SQL units to interact with the database server.
- It consists of SQL executor which parses the input SQL statements and execute the same.

PL/SQL

Some kinds of limitation in SQL:

1) At a time only one Query is executed in SQL.
2) Each Query hits database, if number of hits increased then database performance decreases.
3) Queries are not saved in database. We needs rewrite query again and again.
4) SQL doesn't provide the programmers with a technique of condition checking, looping and branching
5) SQL has no facility of error checking during manipulation of data.

PL/SQL is avoiding limitation of SQL.

Introduction of PL/SQL

The PL/SQL programming language was developed by Oracle Corporation in the late 1980s as "Procedural Language extensions to SQL". This is the extension of Structured Query Language (SQL) that is used in Oracle. PL/SQL allows the programmer to write code in procedural format and the Oracle relational database.

PL/SQL is a collection of user defined object like program, procedure, function, cursor, package, trigger also programming statements and SQL query statement. Example: Online Shopping. PL/SQL can execute multiple queries in form of program.
It combines the data manipulation power of SQL with the processing power of procedural language to create a super powerful SQL queries.

- It provides programming techniques like branching, looping, and condition check.
- It is fully structured programming language.
- It decreases the network traffic because entire block of code is passed to the DBA at one time for execution.
- With PL/SQL, you can use SQL statements to manipulate Oracle data and flow of control statements to process the data.
- PL/SQL is not case sensitive so you are free to use lower case letters or upper case letters except within string and character literals.

⊙ Features of PL/SQL:

- Better performance, as SQL is executed in bulk rather than a single statement.
- Reduces numbers of hits to database.
- Improve database and network performance.
- Enhancebility i.e. existing program enhance for new requirement.
- Reusability i.e. procedure use any number of time
- Modularity i.e. task divide into subtask.
- High Productivity
- Tight integration with SQL
- Full Portability
- Tight Security
- Support Object Oriented Programming concepts.

Comparison between SQL and PL/SQL

SQL(Structured Query Language)	PL/SQL (Programming Language SQL)
SQL is used to execute single query or statement at a time.	PL/SQL is used to execute block of code or program that have multiple statements.

It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
SQL can be used in PL/SQL programs	PL/SQL can't be used in SQL statement.
Mainly used to manipulate data.	Mainly used to create an application.
An example of SQL query: *SELECT * FROM Customers;*	An example of PL/SQL program is: *BEGIN* *dbms_output.put_line('Hello World');* *END;* */*

Structure of PL/SQL

PL/SQL is the procedural approach to SQL in which a **direct instruction** is given to the PL/SQL engine about how to perform actions like storing/fetching/processing data. These instruction are **grouped** together called **Blocks**.

Blocks contain both PL/SQL as well as SQL instruction. All these instruction will be executed as a whole rather than executing a single instruction at a time.

PL/SQL blocks have a pre-defined structure in which the code is to be grouped. Below are different sections of PL/SQL blocks

1. Declaration section
2. Execution section
3. Exception-Handling section

1.Declaration Section

This is the first section of the PL/SQL blocks and its optional part. In this section we can declaration of variables, cursors, exceptions, subprograms, pragma instructions and collections.

- This section can be skipped if no declarations are needed.
- This section starts with the keyword 'DECLARE' for triggers and anonymous block. But in subprograms this keyword will not be present.
- This section should be always followed by execution section.

2.Execution Section

Execution part is the main and mandatory part which actually executes the code that is written inside it. The executable section must have a least one executable statement. i.e. cannot be an empty block.

- This block contains both PL/SQL code and SQL code.
- This can contain one or many blocks inside it as a nested block.
- An executable section starts with the keyword BEGIN and ends with the keyword END or Exception-Handling section.

Syntax of PL/SQL Block Structure:

```
DECLARE       --optional
        <declarations>
        .
        .
        .
BEGIN         --mandatory
        <executable statements. At least one executable statement is mandatory>
        .
        .
        .
EXCEPTION     --optional
        <exception handler>
        .
        .
        .
END;          --mandatory
/
```

3.Exception-Handling Section:

A PL/SQL block has an exception-handling section that starts with the keyword exception. The exception-handling section is where you catch and andle exceptions raised by the code in the execution section. This is an optional section of the PL/SQL blocks.

- Exception raised in the execution block is handled.
- Last part of the PL/SQL block.
- Control from this section can never return to the execution block.
- This section should be always followed by the keyword 'END'.

The Keyword 'END' is the end of PL/SQL block.

Types of PL/SQL block

PL/SQL blocks are of mainly two types.

1. Anonymous blocks
2. Named Blocks

1. Anonymous blocks:

Anonymous blocks are PL/SQL blocks which do not have any names assigned to them. They need to be created and used in the same session because they will not be stored in the server as a database objects. They are written and executed directly, and compilation and execution happen in a single process.

```
DECLARE   (Optional)
            Variable, cursor, user define exception, type.
BEGIN     (Mandatory)
      <Assignment Statement>
      <Sql /plsql statements>
      <Output statement>
EXCEPTION      (Optional)
      <Exception Statements>
END;     (Mandatory)
```

- **Declare Block:** Declaration statement contains in declare block. To declare variables.
  Syntax: Variable_Name data_type (size);
  Example: num int;
  ename varchar2(10);
- **Begin Block**
- **Assignment Block:** store value in the variable
  Variable_name:= value/ expression;
  Ex: num:=123; or select ename into v_ename from emp where eno=12;
- **Data Processing Block:** Any SQL Query use in the program
- **Output Statement:** It used to display output from program or procedure. Oracle provides predefined function.
  dbms_output.put_line('Message' or variable_name);
  Example: dbms_output.put_line('Message Display');
  dbms_output.put_line(Ename);
  dbms_output.put_line('Emp Name:' || Ename);
  (||) used for concatenation values or statements.

- **Exception Statement:** Display user defined or system defined error messages.
- **End:** end of program.

How can I compile and execute

Use (/) operator at the end of program for compilation and execution.

Use 'Set Serveroutput On' statement for print output on screen.

-- use for single line comment.

/* */ for multiline comment.

Example:
```
set serveroutput on
Declare
A int;
Begin
A:=&A;    --& used for take input value from user at runtime.
dbms_output.put_line(A);
end;
/
```

Rules for variable declaration

- Variable name start with alphabet.
- Variable name contains minimum one character and maximum 30 characters.
- It should not allow any space and any special character.
- Keywords are not allowed to declare as a variable name.
- The PL/SQL is not case sensitive so it could be either lowercase or uppercase. For example: v_data and V_DATA refer to the same variables.

2. Named blocks (Procedures, functions, package, trigger):

Named blocks are having a specific and unique name for them. They are stored as the database objects in the server. They can be referred to other object. The compilation process for named blocks happens separately while creating them as a database objects.

Data types in PL/SQL

PL/SQL Data Types

The Oracle PL/SQL Data Types are: Scalar Types (number, character, date, boolean), Composite Types (Collections and Records), Reference Types (Cursor Variables), LOB Types (BFILE, BLOB, CLOB, NCLOB).

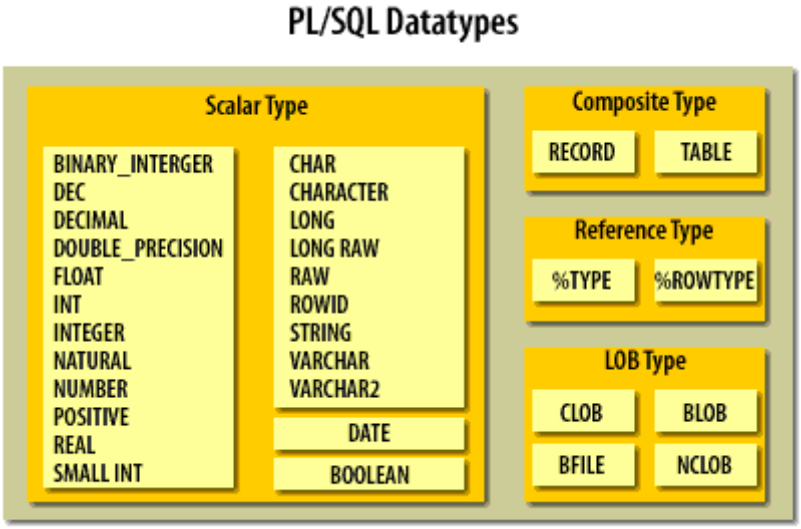
PL/SQL Number Types

BINARY_DOUBLE, BINARY_FLOAT, BINARY_INTEGER, DEC, DECIMAL, DOUBLE PRECISION, FLOAT, INT, INTEGER, NATURAL, NATURALN, NUMBER, NUMERIC, PLS_INTEGER, POSITIVE, POSITIVEN, REAL, SIGNTYPE, SMALLINT

PL/SQL Character and String Types

CHAR, CHARACTER, LONG, LONG RAW, NCHAR, NVARCHAR2, RAW, ROWID, STRING, UROWID, VARCHAR, VARCHAR2

PL/SQL Date, Time and Interval Types
DATE, TIMESTAMP, TIMESTAMP WITH TIMEZONE, TIMESTAMP WITH LOCAL TIMEZONE, INTERVAL YEAR TO MONTH, INTERVAL DAY TO SECOND
PL/SQL Boolean Types
BOOLEAN



1. Scalar: These data types don't include any internal components. It includes data types such as NUMBER, DATE, BOOLEAN, etc.
2. Large Objects (LOB): This type of data type stores objects that are relatively large in size and stored separately from other data types such as text, graphic images, video clips, sound, etc.
3. Composite: These type of data types have internal components that can be accessed individually. It includes records and collections.
4. Reference: As the name sounds, it includes pointers that refer to the location of the other data items.
Now, I have listed down some of the sub data types frequently used in PL/SQL
1. Numeric: Numeric values on which arithmetic operations are performed. It includes sub types such as number, decimal, real, float, etc.
2. Character: Character values on which character operations such as strings are performed. It includes sub types such as char, varchar, varchar2, nvarchar2, etc.
3. Data and Time: This data type is used to store fixed data type which displays and saves time and date values. The default data format saved into the database might be 'DD-MM-YY'. However, you can change and alter the position of the terms accordingly.
4. Boolean: These include logical values on which logical operations are performed. The logical values are the Boolean values TRUE and FALSE and the value NULL. But, SQL has no data type equivalent to BOOLEAN. It cannot be used

in SQL Statements, built in SQL functions such as To_char, PL/SQL function invoked from DQL commands.
5. Number: Syntax: Number(Precision, Scale). Fixed-point or floating-point number with absolute value in range 1E-130 to (but not including) 1.0E126. A NUMBER variable can also represent 0.
6. Float: ANSI and IBM specific floating-point type with maximum precision of 126 binary digits (approximately 38 decimal digits).
7. Integer: ANSI and IBM specific integer type with maximum precision of 38 decimal digits
8. Real: Floating-point type with maximum precision of 63 binary digits (approximately 18 decimal digits).
9. Varchar2: Variable-length character string with maximum size of 32,767 bytes.
10. Rowid: Physical row identifier, the address of a row in an ordinary table.
11. Bfile: Used to store large binary objects in operating system files outside the database System-dependent. Cannot exceed 4GB.
12. Blob: Used to store large binary objects in the database. Memory Capacity: 8 to 128 TB.
13. Clob: Used to store large blocks of character data in the database. Memory Capacity: 8 to 128 TB.
14. Nclob: Used to store large blocks of NCHAR data in the database. Memory Capacity: 8 to 128 TB.

◈ Control Structures

Control structures enable the program to change the logical flow of statements within PL/SQL with a number of control structures. PL/SQL supports two basic programming control structures:
Conditional and iteration branching. These four structure scan be combined in any way to solve a given problem. The following sections cover these structures in more detail.

➢ Conditional Control

You can execute a statement or sequence of statements conditionally with the if-then statement. Sometimes it is necessary to take alternative actions, depending on a condition or circumstance.

1. write a program to add two numbers and display the o/p

```
declare
a number(3):=10;
b number(3):=9;
c number(3);
begin
```

```
c:=a+b;
dbms_output.put_line(c);
end;
/
```

2. Write a program to accept two different numbers and find addition, subtraction, multiplication and division of given number.

```
declare
a number(2):=&a;
b number(2):=&b;
s number(3);
su number(3);
m number(3);
d number(3);
begin
s:=a+b;
su:=a-b;
m:=a*b;
d:=a/b;
dbms_output.put_line('ADDITION '||s);
dbms_output.put_line('SUBSTRACTION '||su);
dbms_output.put_line('MULTIPLICATION '||m);
dbms_output.put_line('DIVITION '||d);
end;
```

3. Write a program to accept empno, ename, sal,DOB,DOJ.Display all the details along with age of the emp, exp of employee, asal,dsal as per current month round to nearest rupees and also display all the allowances (TA+DA+HRA) and gross salary.

```
declare
empno number(3):=&empno;
ename varchar2(10):='&ename';
sal number(10):=&sal;
dob date:='&dob';
doj date:='&doj';
age number(3);
exp number(3);
asal number(5);
dsal number(5,2);
da number(5);
```

```
ta number(5);
hra number(5);
grosal number(15);
begin
age:=months_between(sysdate,dob);
exp:=months_between(sysdate,doj);
asal:=sal*12;
dsal:=sal/to_char(last_day(sysdate),'dd');
da:=sal*10/100;
ta:=sal*20/100;
hra:=sal*5/100;
grosal:=da+ta+hra;
dbms_output.put_line('asal='||asal);
dbms_output.put_line('Exp= '||exp);
dbms_output.put_line('Asal= '||asal);
dbms_output.put_line('dsal= '||dsal);
dbms_output.put_line('Gross sal '||grosal);
dbms_output.put_line('age '||age);
dbms_output.put_line('da '||da);
dbms_output.put_line('TA '||ta);
dbms_output.put_line('hra '||hra);
end;
/
```

- **If-Then Statements**

The simplest form of the conditional control statement is the if-then statement. The syntax
for the if-then statement is as follows:

```
if condition then
statement(s);
end if;
```

The statements that immediately follow the conditional if statement are executed only if the condition evaluates to True. If the condition evaluates to False, the statements are not processed and are passed over.

- **If-Then-Else Statements**

You will have different sets of statements to execute depending on the conditional statement. The if-then-else statement provides a means to always execute statements in a conditional statement.
The syntax of the if-then-else structure is very similar to the if-then syntax. The following Example uses the if-then-else structure:

```
if max_bal> 100000 then
updatemst_audit -- executed only if condition is true
set comments = ‘sell no more’;
else
updatemst_audit -- executed only if condition is false
set comments = ‘sell more’;
end if.
```

4. Write a program to accept two numbers find the smaller no.

```
declare
a number(3):=&a;
b number(3):=&b;
begin
if a>b then
dbms_output.put_line(a||' is the bigger one');
else if a=b then
dbms_output.put_line(a ||' and ' ||b|| 'are equal');
else
dbms_output.put_line(b||' is the bigger one');
end if;
end if;
end;
/
```

5 write a program to accept two date value and find the recent one

```
declare
a date:='&a';
b date:='&b';
begin
if a<b then
dbms_output.put_line(a||' is previous date');
else
dbms_output.put_line(b||' is previous date');
end if;
end;
/
```

6.write a program to accept two different date and find the recent day of the week.

```
declare
a date:='&a';
b date:='&b';
```

```
begin
if (to_char(a,'d')>to_char(b,'d')) then
dbms_output.put_line(a ||' is recent day');
else
dbms_output.put_line(b ||' is recent day');
end if;
end;
```

7.w.a.p to accept name,sid,marks1,marks2,marks3 based on avg declare the result. If marks of any one subject are less than 35 declare as fail.

```
declare
name varchar2(10):='&name';
sid number(4):=&sid;
marks1 number(4):=&marks1;
marks2 number(4):=&marks2;
marks3 number(4):=&marks3;
avg number(3,2);
begin
avg:=(marks1+marks2+marks3/3);
if (avg>=70) then
dbms_output.put_line('DISTICTION');
elsif (avg>=60 and avg<70) then
dbms_output.put_line('FIRST division');
elsif (avg>=50 and avg<60) then
dbms_output.put_line('2nd division');
elsif (avg>=35 and avg<50) then
dbms_output.put_line('3rd division');
else
dbms_output.put_line('FAIL');
end if;
end;
/
```

8.write a program to accept the product id,productdesc,quanlity and unit price .Now calculate the bill.If the bill more than 500 & less than 1500 give a discount of 5%,if bill is more than 1500 then 10% discount. Display all the details along with the bill amount,discount,net payable.

```
declare
pid number(4):=&pid;
```

```
pdesc varchar2(10):='&pdesc';
quan number(5):=&quan;
unit_pr number(6,2):=&unit_pr;
bill number(8,2);
disc number(5,2);
net_pay number(8,2);
begin
bill:=quan*unit_pr;
if bill>1500 then
disc:=bill*10/100;
net_pay:=bill-disc;
else if (bill<1500 and bill>500) then
        disc:=bill*5/100;
        net_pay:=bill-disc;
else disc:=0;
end if;
end if;
dbms_output.put_line('pid '||pid);
dbms_output.put_line('bill '||bill);
dbms_output.put_line('discount '||disc);
dbms_output.put_line('Net '||net_pay);
end;
/
```

- **Iterative Control**

Iterative control statements enable you to execute a sequence of statements multiple times. The simplest form of an iterative statement is the loop.

➢ **Loop**

Loop is execution of a sequence of statements repeatedly.

Loop Statements. The syntax for the loop statement is

```
loop
statement(s)
end loop;
```

➢ **Types of Loop**

1. Simple loop
2. While loop
3. for loop
4. Reverse for loop

SIMPLE LOOP

To Print Numbers

```
Declare
A number:=1;
Begin
Loop
Exit when a>20;
Dbms_output.put_line(a);
A:=a+1;
End loop;
End;
/
```

REVERSE NO:

```
 Declare
A number:=100;
Begin
Loop
Exit when a < 1;
Dbms_output.put_line(a);
A:=a - 1;
End loop;
End;
```

To Print The No Is Prime Or Not:

```
Declare
A number:=&no;
B number;
C number:=0;
I number:=1;
Begin
Loop
Exit when i> a;
B:=mod(a,i);
If b=0 then
C:=c+1;
End if;
I:=i+1;
End loop;
If c=2 then
```

```
Dbms_output.put_line('prime');
Else
Dbms_output.put_line('not prime');
End if;
End;
/
```

➢ **While loop**

The while-loop structure repeats a statement until a condition is no longer true. The syntax for the while loops is as follows:

```
while condition loop
statement(s)
end loop
```

9. write a program to print a series of numbers from that no to specified no.

```
declare
n number(3):=&n;
a number(3):=&a;
begin
while(a<=n)
loop
dbms_output.put_line(a);
a:=a+1;
end loop;
end;
/
```

10.w.a p accept a number and print all the even numbers from 1 upto given number.

```
declare
a number(3):=&a;
b number(3):=1;
c number(4);
begin
while (b<=a)
loop
c:=mod(b,2);
if c=0 then
dbms_output.put_line(b);
```

```
end if;
b:=b+1;
end loop;
end;
```

To Print The No is Even Or Odd

```
Declare
No number:=&no;
Begin
If  mod(no,2)=0 then
Dbms_output.put_line(' the no is even no');
Else
Dbms_output.put_line(' the no is odd no');
End if;
End;
```

To Print The No Is Prime Or Not

```
Declare
A number:=&no;
B number;
C number:=0;
I number:=1;
Begin
While i<= a loop
B:=mod(a,i);
If b=0 then
C:=c+1;
End if;
I:=i+1;
End loop;
If c=2 then
Dbms_output.put_line('prime');
Else
Dbms_output.put_line('not prime');
End if;
End;
/
```

***For* Loop Statements**

For loop is a mechanism to cycle through a loop a fixed number of times. A for loop iterates over and over for a specified number of times. The syntax for the for loop is as follows:

```
for counter in lower-bound..upper-bound loop
statement(s)
end loop;
```

w.a.p to accept a number and to print multiplication table of that number.

```
declare
a number(4):=&a;
b number(3);
begin
fori in 1..10
loop
b:=a*i;
dbms_output.put_line(a||'*'||i||'='||b);
end loop;
end;
/
```

to print factorial of given number

```
declare
n number(3):=&n;
fact number(5):=1;
begin
fori in 1..n
loop
fact:=fact*i;
end loop;
dbms_output.put_line('THE FACTORIAL OF THE GIVEN NUMBER '||fact);
end;
/
```

To Print The No Is Prime Or Not

```
DECLARE
 A NUMBER:=&N;
 B NUMBER:=0;
 C NUMBER:=0;
 BEGIN
 FOR I IN 1..A LOOP
 B:=MOD(A,I);
```

```
IF B=0 THEN
C:=C+1;
END IF;
END LOOP;
IF C=2 THEN
DBMS_OUTPUT.PUT_LINE('PRIME');
ELSE
DBMS_OUTPUT.PUT_LINE('NOT PRIME');
END IF;
END;/
```

Reverse for loop

```
Begin
For i in reverse 1..10 loop
Dbms_output.put_line(i);
End loop;
End;
```

Loop inside loop

```
Declare
A number:=1;
Begin
Loop
Exit when a>=10;
For i in 1..10 loop
Dbms_output.put_line(a||'*'||i||' '||'='||a*i);
End loop;
A:=a+1;
End loop;
End;
```

PL/SQL GOTO Statement

In PL/SQL, GOTO statement makes you able to get an unconditional jump from the GOTO to a specific executable statement label in the same subprogram of the PL/SQL block.Here the label declaration which contains the label_name encapsulated within the << >> symbol and must be followed by at least one statement to execute.

Syntax:

GOTO label_name;

Here the label declaration which contains the label_name encapsulated within the << >> symbol and must be followed by at least one statement to execute.

```
GOTO label_name;
  ..
  ..
<<label_name>>
Statement;
```

```
BEGIN
  GOTO second_message;
  <<first_message>>
  DBMS_OUTPUT.PUT_LINE( 'Hello' );
  GOTO the_end;
  <<second_message>>
  DBMS_OUTPUT.PUT_LINE( 'PL/SQL GOTO Demo' );
  GOTO first_message;
  <<the_end>>
  DBMS_OUTPUT.PUT_LINE( 'and good bye...' );
END;
```

Restriction on GOTO statement

Following is a list of some restrictions imposed on GOTO statement.

- Cannot transfer control into an IF statement, CASE statement, LOOP statement or sub-block.
- Cannot transfer control from one IF statement clause to another or from one CASE statement WHEN clause to another.
- Cannot transfer control from an outer block into a sub-block.
- Cannot transfer control out of a subprogram.
- Cannot transfer control into an exception handler.

```
DECLARE
  n_sales NUMBER;
  n_tax NUMBER;
BEGIN
    GOTO inside_if_statement;
    IF n_sales > 0 THEN
      <<inside_if_statement>>
      n_tax  := n_sales * 0.1;
    END IF;
END;
```

PL/SQL EXIT Statement

The **EXIT** statement in PL/SQL programming language has the following two usages –

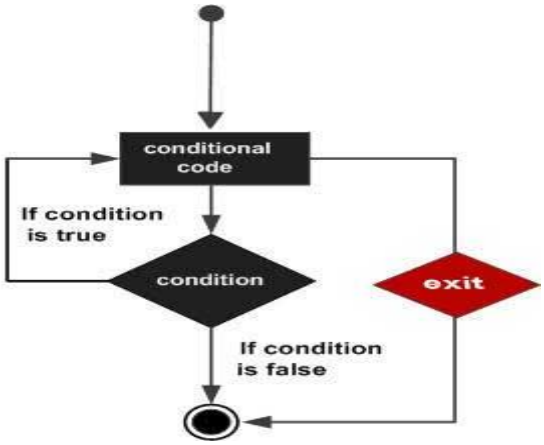
- When the EXIT statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- If you are using nested loops (i.e., one loop inside another loop), the EXIT statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for an EXIT statement in PL/SQL is as follows –

```
EXIT;
```

Flow Diagram



Example

```
DECLARE
  a number(2) := 10;
BEGIN
  -- while loop execution
  WHILE a < 20 LOOP
    dbms_output.put_line ('value of a: ' || a);
    a := a + 1;
    IF a > 15 THEN
      -- terminate the loop using the exit statement
      EXIT;
    END IF;
  END LOOP;
```

```
END;
```

The EXIT WHEN Statement

The **EXIT-WHEN** statement allows the condition in the WHEN clause to be evaluated. If the condition is true, the loop completes and control passes to the statement immediately after the END LOOP.

Following are the two important aspects for the EXIT WHEN statement −

- Until the condition is true, the EXIT-WHEN statement acts like a NULL statement, except for evaluating the condition, and does not terminate the loop.
- A statement inside the loop must change the value of the condition.

Syntax

The syntax for an EXIT WHEN statement in PL/SQL is as follows −

EXIT WHEN condition;

The EXIT WHEN statement **replaces a conditional statement like if-then** used with the EXIT statement.

Example

```
DECLARE
  a number(2) := 10;
BEGIN
  -- while loop execution
  WHILE a < 20 LOOP
    dbms_output.put_line ('value of a: ' || a);
    a := a + 1;
    -- terminate the loop using the exit when statement
  EXIT WHEN a > 15;
  END LOOP;
END;
/
```

CASE Statement

- A CASE statement is similar to IF-THEN-ELSIF statement that selects one alternative based on the condition from the available options.
- CASE statement uses “selector” rather than a Boolean expression to choose the sequence.
- The value of the expression in the CASE statement will be treated as a selector.
- The expression could be of any type (arithmetic, variables, etc.)

- Each alternative is assigned with a certain pre-defined value (selector), and the alternative with selector value that matches the conditional expression value will get executed.
- Unlike IF-THEN-ELSIF, the CASE statement can also be used in SQL statements.

ELSE block in CASE statement holds the sequence that needs to be executed when none of the alternatives got selected

```
CASE [expression]
   When condition_1 then result_1
   When condition_2 then result_2
    .
    .
    .
   When condition_n then result_n
Else
    Result
End
```

```
declare
   grade varchar(30);
 Begin
   grade:='&grade';
   case grade
     when 'A' then dbms_output.put_line('Excellent');
     when 'A' then dbms_output.put_line('Very Good');
     when 'A' then dbms_output.put_line('Good');
     when 'A' then dbms_output.put_line('Fair');
    else
      dbms_output.put_line('No such grade');
   end case;
 end;
```

Reference Type

Datatype	Description
%Type	This data type is used to store value unknown data type column in a table. The column is identified by %type data type. Eg. emp.eno%type emp name is a table, eno is an unknown data type column and %Type is data type to hold the value.
%RowType	This data type is used to store values unknown data type in all columns in a table. All columns are identified by %RowType datatype. Eg. emp%rowtype emp name is a table, all column type is %rowtype.
%RowID	RowID is data type. RowID is two types extended or restricted. Extended return 0 and restricted return 1 otherwise return the row number.

%TYPE	%ROWTYPE
%TYPE is used to declare a variable with the same data type as an existing database column or variable.	%ROWTYPE is used to declare a variable that can hold an entire row of a database table or a database cursor.
It is used when you want to create a variable that has the same data type as a column in a table or another variable.	It is used to declare a variable that can store an entire row from a table or cursor.
For example, if you have a table named "Employee" with a column named "EmpID" of type NUMBER, you can declare a variable named "EmpNo" with the same data type as the "EmpID" column by using the following syntax: EmpNo Employee.EmpID%TYPE;	For example, if you have a table named "Employee" you can declare a variable named "EmpRec" that can store an entire row from the Employee table using the following syntax: EmpRec Employee%ROWTYPE;

Into form of select statement

- The SELECT INTO statement retrieves values from one or more database tables (as the SQL SELECT statement does) and stores them in variables.
- 'SELECT' statement should return only one record while using 'INTO' clause as one variable can hold only one value.
- If the 'SELECT' statement returns more than one value than 'TOO_MANY_ROWS' exception will be raised.
- 'SELECT' statement will assign the value to the variable in the 'INTO' clause, so it needs to get at least one record from the table to populate the value.
- If it didn't get any record, then the exception 'NO_DATA_FOUND' is raised.
- The number of columns and their datatype in 'SELECT' clause should match with the number of variables and their datatypes in the 'INTO' clause.
- The values are fetched and populated in the same order as mentioned in the

```
SQL> desc std;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLLNO                                    NOT NULL NUMBER
 NAME                                               VARCHAR2(10)
 ADDRESS                                            VARCHAR2(30)
```

```
SQL> declare
  2  r std.rollno%type;
  3  n std.name%type;
  4  a std.address%type;
  5     begin
  6       select rollno,name,address into r,n,a from std where rollno=1;
  7       dbms_output.put_line(r);
  8       dbms_output.put_line(n);
  9       dbms_output.put_line(a);
 10     end;
 11  /

PL/SQL procedure successfully completed.

SQL> /
1
aaa

PL/SQL procedure successfully completed.
SQL>  declare
  2     r std%rowtype;
  3      begin
  4       select * into r from std where rollno=1;
  5       dbms_output.put_line(r.rollno);
  6       dbms_output.put_line(r.name);
  7       dbms_output.put_line(r.address);
  8     end;
  9  /
1
aaa

PL/SQL procedure successfully completed.
```

```
declare
  r std.rollno%type;
  n std.name%type;
  a std.address%type;
begin
  r:=&r;
  n:=&n;
  a:=&a;
    insert into std values(r,n,a);
end;
```